

開始使用獨立元件

來源：<https://codelabs.developers.google.com/angular-standalone-components?hl=zh-tw>

步驟數：11

在這個適合初學者的程式碼研究室中，您將瞭解如何使用 Angular v14 獨立元件建構範例應用程式

目錄

1. [1. 簡介](#)
2. [2. 環境設定](#)
3. [3. 建立新應用程式](#)
4. [4. 顯示相片](#)
5. [5. 新增獨立元件](#)
6. [6. 新增表單的 UI](#)
7. [7. 在表單中新增事件處理常式](#)
8. [8. 新增即時通訊對話](#)
9. [9. 將應用程式部署至 Google App Engine](#)
10. [10. 使用 Express.js 架構部署應用程式](#)
11. [11. 恭喜](#)

步驟 1 · 約 0 分鐘

1. 簡介

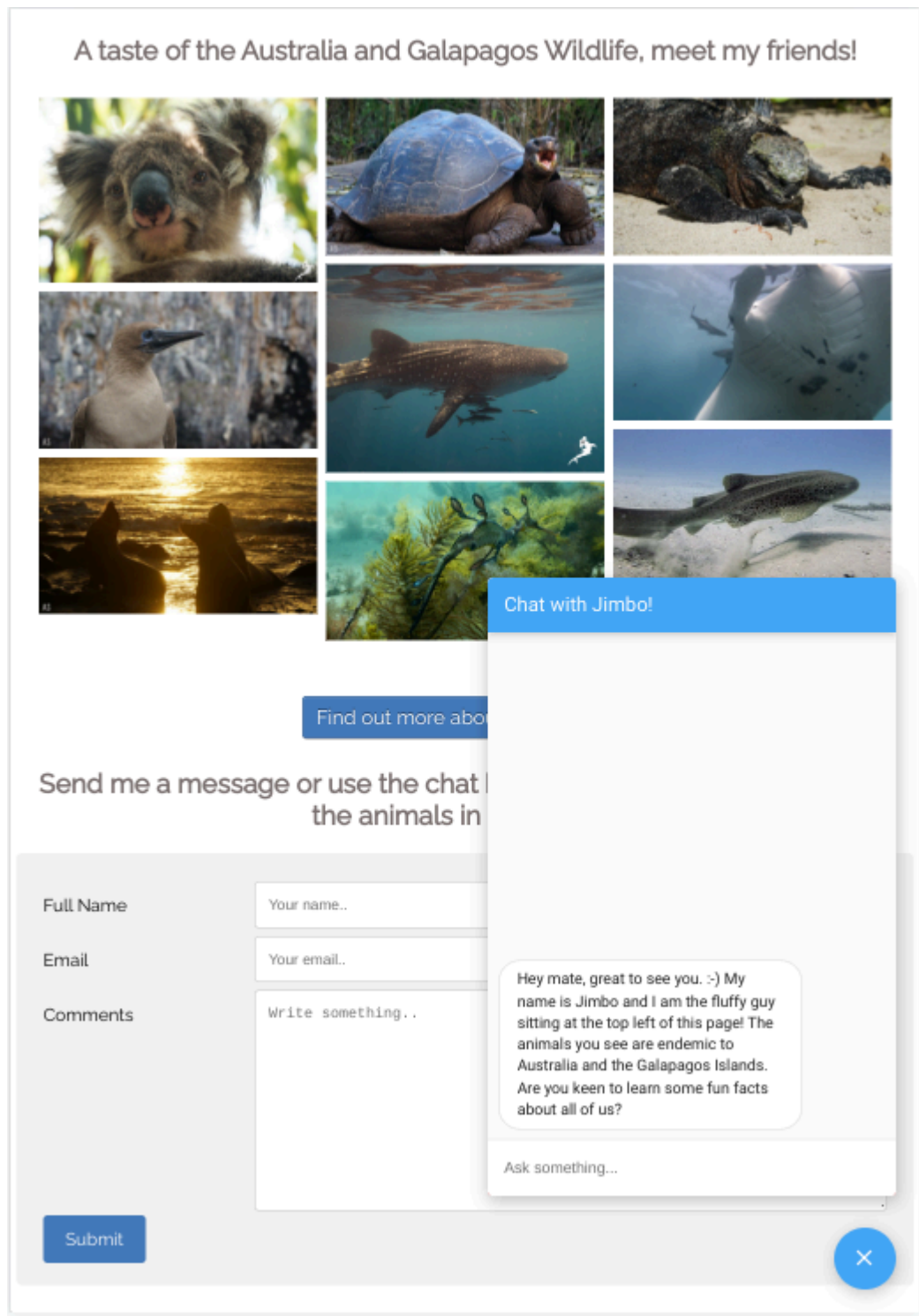
建構目標

在本程式碼研究室中，您將使用 Angular v14 建構野生動物相片庫應用程式。完成申請後，系統會顯示一組相片，並提供傳送訊息表單，方便你聯絡攝影師，以及聊天視窗，讓你瞭解相片中動物的有趣資訊。

您可以使用 Angular v14 和新的獨立元件功能，在應用程式中建構所有項目。

所有 Angular 架構和 Angular CLI 的參考資料都反映了 Angular v14。獨立元件是 Angular 第 14 版的預先發布功能，因此您必須使用 Angular 第 14 版建立全新應用程式。獨立元件提供簡化的 Angular 應用程式建構方式。獨立元件、獨立指令和獨立管道旨在減少 `NgModules` 的需求，進而簡化撰寫體驗。獨立元件可充分運用現有的 Angular 程式庫生態系統。

這是您今天建構的應用程式。



學習目標

- 如何使用 Angular CLI 搭建新專案
- 如何使用 Angular 獨立元件，簡化 Angular 應用程式的開發作業
- 如何建立獨立元件，也就是如何建構 UI 並新增一些商業邏輯
- 如何使用獨立元件啟動應用程式
- 如何延遲載入獨立元件
- 如何使用 Dialogflow Messenger，在獨立元件中嵌入即時通訊對話
- 如何使用 Google Cloud CLI (gcloud) 將 Angular 應用程式部署至 Google Cloud App Engine

軟硬體需求

- Gmail 或 Google Workspace 帳戶
 - 對下列主題有基本瞭解：
 - HTML。如何建立元素。
 - CSS 和更少。如何使用 CSS 選擇器及建立樣式定義檔。
 - TypeScript 或 JavaScript。如何與 DOM 結構互動。
 - git 和 GitHub。如何建立存放區的分支並複製存放區。
 - 指令列介面，例如 `bash` 或 `zsh`。如何瀏覽目錄及執行指令。
-

2. 環境設定

設定本機環境

如要完成本程式碼研究室，請務必在本機電腦上安裝下列軟體。

- [Node.js 的有效 LTS 或維護 LTS 版本](#)。用於安裝 Angular 架構和 Angular CLI。

節點版本	Angular 支援
14.15 以上版本	有權限
16.10 以上版本	有權限
18.1.0	不支援

如要確認本機電腦上的 Node.js 版本，請從指令列視窗執行下列 `node` 指令。

```
node -v
```

- 程式碼編輯器或 IDE。用於開啟及編輯檔案。[Visual Studio Code](#) 或您選擇的其他程式碼編輯器。

安裝 Angular CLI

設定所有依附元件後，請執行下列 `npm` 指令，從指令列視窗安裝 Angular CLI。

```
npm install --global @angular/cli
```

如要確認設定是否正確，請從指令列執行下列 Angular 指令。

```
ng version
```

如果 Angular 指令順利完成，您應該會收到類似以下螢幕截圖的訊息。



```
Angular CLI: 14.2.5
Node: 16.13.2
Package Manager: npm 8.19.2
OS: darwin x64
```

```
Angular:
```

```
...
```

Package	Version
@angular-devkit/architect	0.1402.5 (cli-only)
@angular-devkit/core	14.2.5 (cli-only)
@angular-devkit/schematics	14.2.5 (cli-only)
@schematics/angular	14.2.5 (cli-only)

原始碼和圖片

您將從頭建立整個應用程式，而這個逐步程式碼研究室會提供協助。請注意，GitHub 存放區包含最終程式碼。如果遇到困難，請檢查最終程式碼和圖片庫頁面顯示的圖片。

下載原始碼。

1. 在瀏覽器中前往下列頁面。

```
https://github.com/angular/codelabs/tree/standalone-components
```

2. 在指令列視窗中，分叉並複製存放區。

在下一個步驟中，請建構相片庫應用程式。

步驟 3 · 約 0 分鐘

3. 建立新應用程式

如要建立初始的入門應用程式，請完成下列動作。

使用 Angular CLI 建立新的工作區。如要進一步瞭解 Angular CLI 和 Angular 架構，請前往 angular.io。

建立新專案

在指令列視窗中，完成下列動作。

1. 輸入下列 Angular 指令，建立名為 `photo-gallery-app` 的新 Angular 專案。

```
ng new photo-gallery-app
```

2. 在每個提示中接受預設選項。Angular 架構會安裝必要的套件和依附元件。這個程序可能需要幾分鐘。

Angular CLI 完成後，您會擁有新的 Angular 工作區和簡單的應用程式，可隨時執行。

新應用程式的結構與標準 Angular 應用程式類似。在本程式碼研究室中，新應用程式的 `NgModule` 是多餘的。

移除應用程式模組

如要移除應用程式模組，請完成下列動作。

1. 前往新 `photo-gallery-app` 專案目錄中的 `src/app` 目錄。
2. 刪除 `app.module.ts` 檔案。

刪除應用程式模組後，應用程式中就不會再有任何模組。您的應用程式只有一個元件，也就是應用程式元件。您必須將該元件宣告為獨立元件。

宣告獨立元件

在程式碼編輯器中完成下列動作。

1. 前往新 `photo-gallery-app` 專案目錄中的 `src/app` 目錄。
2. 開啟 `app.component.ts` 檔案。
3. 將下列參數和值新增至裝飾器清單。

```
standalone: true
```

`app.component.ts` 檔案應與下列程式碼範例非常相似。


```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css'],
  standalone: true
})
export class AppComponent {
  title = 'photo-gallery-app';
}
```

4. 儲存 `app.component.ts` 檔案。

編譯新的獨立應用程式

在指令列視窗中，完成下列動作。

1. 前往新的 `photo-gallery-app` 專案目錄
2. 輸入下列 Angular 指令，編譯新應用程式。

```
ng serve
```

您的應用程式應該無法編譯。別擔心，你只需要修正幾項問題。

使用 bootstrapApplication API

如要讓應用程式在沒有 `NgModule` 的情況下執行，您必須使用 `bootstrapApplication` API，將獨立元件做為根元件。

移除應用程式模組的參照

在程式碼編輯器中完成下列動作。

1. 前往新 `photo-gallery-app` 專案目錄中的 `src` 目錄
2. 開啟 `main.ts` 檔案。
3. 移除下列匯入程式碼，因為您不再有應用程式模組。

```
import { AppModule } from './app/app.module';
import { platformBrowserDynamic } from '@angular/platform-browser-dynamic';
```

4. 移除下列 Bootstrap 程式碼，因為您不再有應用程式模組。

```
platformBrowserDynamic().bootstrapModule(AppModule)
  .catch(err => console.error(err));
```

`main.ts` 檔案應與下列程式碼範例相符。

```
import { enableProdMode } from '@angular/core';
import { environment } from '../environments/environment';

if (environment.production) {
  enableProdMode();
}
```

新增 bootstrapApplication 元件

在程式碼編輯器中完成下列動作。

1. 前往新 `photo-gallery-app` 專案目錄中的 `src` 目錄。
2. 開啟 `main.ts` 檔案。
3. 從 `@angular/platform-browser` 服務匯入 `bootstrapApplication` 元件。

```
import { bootstrapApplication } from '@angular/platform-browser';
```

4. 加入下列程式碼來啟動應用程式。

```
bootstrapApplication(AppComponent).catch(err => console.error(err));
```

5. 匯入元件和必要程式庫。

```
import { AppComponent } from '../app/app.component';
```

`main.ts` 檔案應與下列程式碼範例相符。

```
import { enableProdMode } from '@angular/core';
import { bootstrapApplication } from '@angular/platform-browser';
import { AppComponent } from '../app/app.component';
import { environment } from '../environments/environment';

if (environment.production) {
  enableProdMode();
}

bootstrapApplication(AppComponent).catch(err => console.error(err));
```

新增路由器和通用模組

如要使用 Router 和其他常見模組功能，您必須將每個模組直接匯入元件。

在程式碼編輯器中完成下列動作。

1. 前往新 `photo-gallery-app` 專案目錄中的 `src/app` 目錄。
2. 開啟 `app.component.ts` 檔案。
3. 將必要模組匯入元件。

```
import { CommonModule } from '@angular/common';
import { RouterModule } from '@angular/router';
```

4. 在元件中新增匯入項目。

```
imports: [CommonModule, RouterModule],
```

`app.component.ts` 檔案應與下列程式碼範例相符。

```
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';
import { RouterModule } from '@angular/router';

@Component({
  selector: 'app-root',
  imports: [CommonModule, RouterModule],
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css'],
  standalone: true
})
export class AppComponent {
  title = 'photo-gallery-app';
}
```

5. 儲存 `app.component.ts` 檔案。

編譯並執行新的獨立應用程式

在指令列視窗中，完成下列動作。

1. 前往新的 `photo-gallery-app` 專案目錄。
2. 輸入下列 Angular 指令，編譯及執行應用程式，並開啟網路伺服器。您可能需要先關閉 IDE，再執行應用程式，才能移除所有編譯錯誤。

```
ng serve
```

開發伺服器應在通訊埠 `4200` 上執行。所有先前的錯誤都應已消失，且編譯作業應已成功。非常好，您已成功建立 Angular 應用程式，該應用程式不含模組，且具有獨立元件。

1. 接下來，您要美化應用程式，顯示一些相片。
-

步驟 4 · 約 0 分鐘

4. 顯示相片

新應用程式的設計目的是做為相片庫，因此應顯示一些相片。

元件是 Angular 應用程式的核心建構區塊。元件有 3 個主要層面。

- 範本的 HTML 檔案
- 樣式的 CSS 檔案
- 應用程式行為的 TypeScript 檔案

將相片移至應用程式

圖片位於您先前從 GitHub 下載的應用程式中。

1. 前往 GitHub 專案的 `src/assets` 目錄。
2. 將檔案複製並貼到 `photo-gallery-app` 專案目錄的 `analogue` 目錄中。

建立 HTML 範本

`app.component.html` 檔案是與 `AppComponent` 元件相關聯的 HTML 範本檔案。

在程式碼編輯器中完成下列動作。

1. 前往新 `photo-gallery-app` 專案目錄中的 `src/app` 目錄。
2. 開啟 `app.component.html` 檔案。
3. 刪除所有現有的 HTML。
4. 複製下列程式碼範例中的 HTML。

```
<article>
  <h1>Above and below the Ocean</h1>
  <h2>A taste of the Australia and Galapagos Wildlife, meet my friends!</h2>
  <section>
    <section class="row">
      <div class="column">
        
        
        
      </div>
      <div class="column">
        
        
        
      </div>
      <div class="column">
        
        
        
      </div>
    </section>
  </section>
</article>
```

5. 儲存 `app.component.html` 檔案。

建立樣式定義檔

在程式碼編輯器中完成下列動作。

1. 前往新 `photo-gallery-app` 專案目錄中的 `src` 目錄。
2. 開啟 `styles.css` 檔案。
3. 複製下列程式碼範例中的 CSS。

```
article {
  display: flex;
  justify-content: center;
  align-items: center;
  flex-direction: column;
  font-family: 'Raleway';
}

h1 {
  color: #4479BA;
  text-align: center;
  font-size: xx-large;
}

h2 {
  color: rgb(121, 111, 110);
  text-align: center;
}

.row {
  display: flex;
  flex-wrap: wrap;
  padding: 0 4px;
}

/* Create four equal columns that sits next to each other */
.column {
  flex: 25%;
  padding: 0 4px;
}

.column img {
  margin-top: 8px;
  vertical-align: middle;
  width: 100%;
}

.link_button {
  -webkit-border-radius: 4px;
  -moz-border-radius: 4px;
  border-radius: 4px;
  border: solid 1px #20538D;
  text-shadow: 0 -1px 0 rgba(0, 0, 0, 0.4);
  -webkit-box-shadow: inset 0 1px 0 rgba(255, 255, 255, 0.4), 0 1px 1px rgba(0, 0, 0, 0.2);
  -moz-box-shadow: inset 0 1px 0 rgba(255, 255, 255, 0.4), 0 1px 1px rgba(0, 0, 0, 0.2);
  box-shadow: inset 0 1px 0 rgba(255, 255, 255, 0.4), 0 1px 1px rgba(0, 0, 0, 0.2);
  background: #4479BA;
  color: #FFF;
  padding: 8px 12px;
  text-decoration: none;
  margin-top: 50px;
}
```

```
    font-size: large;
  }

  @media screen and (max-width: 800px) {
    .column {
      flex: 50%;
      max-width: 50%;
    }
  }

  @media screen and (max-width: 600px) {
    .column {
      flex: 100%;
      max-width: 100%;
    }
  }
}
```

4. 儲存 `styles.css` 檔案

更新索引檔案

在程式碼編輯器中完成下列動作。

1. 前往新 `photo-gallery-app` 專案目錄中的 `src` 目錄。
2. 開啟 `index.html` 檔案。
3. 新增 `Raleway` 字型，讓所有頁面都能沿用。

```
<link rel="stylesheet" href="https://fonts.googleapis.com/css?family=Raleway">
```

4. 儲存 `index.html` 檔案。
 5. 儲存程式碼並檢查瀏覽器。開發伺服器執行時，儲存變更後，瀏覽器就會反映變更。
 6. 接下來，您要建立新的獨立元件，以便傳送意見回饋及與 Jimbo 對話。請繼續完成這個程式碼研究室，進一步瞭解 Jimbo。
-

5. 新增獨立元件

如您目前所見，獨立元件可減少對 NgModule 的需求，簡化 Angular 應用程式的建構方式。在接下來的章節中，您將建立新的獨立元件，讓使用者傳送意見回饋並與虛擬代理程式對話。

建立新的獨立元件

如要建立這個新元件，請再次使用 Angular CLI。

在指令列視窗中，完成下列動作。

1. 前往新的 `photo-gallery-app` 專案目錄。
2. 輸入下列 Angular 指令，建立名為 `feedback-and-chat` 的新元件。

```
ng generate component feedback-and-chat --standalone
```

下表說明指令的各個部分。

配件	詳細資料
<code>ng</code>	定義 Angular 架構的所有 Angular CLI 指令
<code>generate component</code>	建立新元件的架構
<code>feedback-and-chat</code>	元件名稱
<code>--standalone</code>	告知 Angular 架構建立獨立元件

匯入新的獨立元件

如要使用新的獨立元件，請先將其新增至 `app.components.ts` 檔案中的 `imports` 陣列。

這是獨立元件，因此您只需匯入，就像匯入模組一樣。

在程式碼編輯器中完成下列動作。

1. 前往新 `photo-gallery-app` 專案目錄中的 `src/app` 目錄。
2. 開啟 `app.component.ts` 檔案。
3. 匯入新的獨立元件。

```
import { FeedbackAndChatComponent } from '../feedback-and-chat/feedback-and-chat.component';
```

4. 更新元件中的匯入項目。

```
imports: [CommonModule, RouterModule, FeedbackAndChatComponent],
```


`app.component.ts` 檔案應與下列程式碼範例相符。

```
import { CommonModule } from '@angular/common';
import { Component } from '@angular/core';
import { RouterModule } from '@angular/router';
import { FeedbackAndChatComponent } from '../feedback-and-chat/feedback-and-chat.component';

@Component({
  selector: 'app-root',
  imports: [CommonModule, RouterModule, FeedbackAndChatComponent],
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css'],
  standalone: true
})
export class AppComponent {
  title = 'photo-gallery-app';
}
```

5. 儲存 `app.component.ts` 檔案。

延遲載入元件

從急切載入範例切換至延遲載入範例，這樣一來，程式碼就不會傳送至用戶端，直到您需要時才會傳送。延遲載入是減少網頁載入時間、提升效能及改善使用者體驗的絕佳方法。路由器會處理延遲載入，這與 `ngModule` 和獨立元件相同。

更新與應用程式元件相關聯的 HTML 範本

如要延遲載入獨立元件，請在 UI 中新增按鈕，只有在使用者選取時才啟用元件。

在程式碼編輯器中完成下列動作。

1. 前往新 `photo-gallery-app` 專案目錄中的 `src/app` 目錄。
2. 開啟 `app.component.html` 檔案。
3. 捲動至檔案結尾，然後在關閉 `article` 元素前新增下列程式碼範例。

```
<a class="link_button" routerLink="feedback-and-chat">Find out more about these guys!</a>
<router-outlet></router-outlet>
```

4. `app.component.html` 檔案應與下列程式碼範例相符。

```

<article>
  <h1>Above and below the Ocean</h1>
  <h2>A taste of the Australia and Galapagos Wildlife, meet my friends!</h2>
  <section>
    <section class="row">
      <div class="column">
        
        
        
      </div>
      <div class="column">
        
        
        
      </div>
      <div class="column">
        
        
        
      </div>
    </section>
  </section>
  <a class="link_button" routerLink="feedback-and-chat">Find out more about these guys!
</a>
  <router-outlet></router-outlet>
</article>

```

5. 儲存 `app.component.html` 檔案。

設定路線

在程式碼編輯器中完成下列動作。

1. 前往新 `photo-gallery-app` 專案目錄中的 `src` 目錄。
2. 開啟 `main.ts` 檔案。
3. 匯入 `provideRouter` 方法和 `Routes` 模組。Angular 14.2.0 版導入了新的 `provideRouter` 方法，可讓我們為應用程式設定一組路徑。

```
import { provideRouter, Routes } from '@angular/router';
```

4. 複製下列程式碼片段，並貼到匯入項目和 `if` 陳述式之間。

```

const routes = [
  {
    path: 'feedback-and-chat',
    loadComponent: () => import('./app/feedback-and-chat/feedback-and-
chat.component').then(c => c.FeedbackAndChatComponent),
  }
]

```

5. 複製並貼上下列程式碼片段，然後取代 `bootstrapApplication` 方法。

```
bootstrapApplication(AppComponent, {
  providers: [
    provideRouter(routes)
  ]
}).catch(err => console.error(err));
```

`main.ts` 檔案應與下列程式碼範例相符。

```
import { enableProdMode } from '@angular/core';
import { bootstrapApplication } from '@angular/platform-browser';
import { provideRouter, Routes } from '@angular/router';
import { AppComponent } from './app/app.component';
import { environment } from './environments/environment';

const routes = [
  {
    path: 'feedback-and-chat',
    loadComponent: () =>
      import('./app/feedback-and-chat/feedback-and-chat.component').then(c =>
        c.FeedbackAndChatComponent),
  }
]

if (environment.production) {
  enableProdMode();
}

bootstrapApplication(AppComponent, {
  providers: [
    provideRouter(routes)
  ]
}).catch(err => console.error(err));
```

6. 儲存 `main.ts` 檔案。

使用 Chrome 開發人員工具編譯及檢查

使用 Chrome 開發人員工具，查看 Angular 架構如何延遲載入元件。

在指令列視窗中，完成下列動作。

1. 前往新的 `photo-gallery-app` 專案目錄。
2. 輸入下列 Angular 指令，編譯及執行應用程式，並開啟網路伺服器。

```
ng serve
```

開發伺服器應在通訊埠 `4200` 上執行。

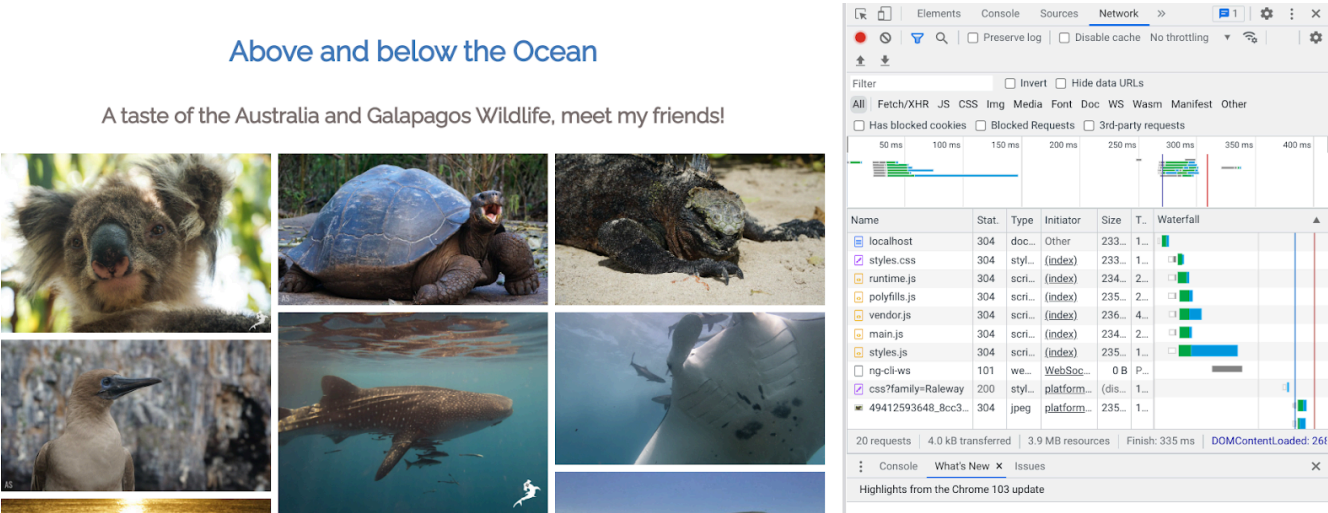
在瀏覽器中完成下列動作。

1. 前往下列頁面。

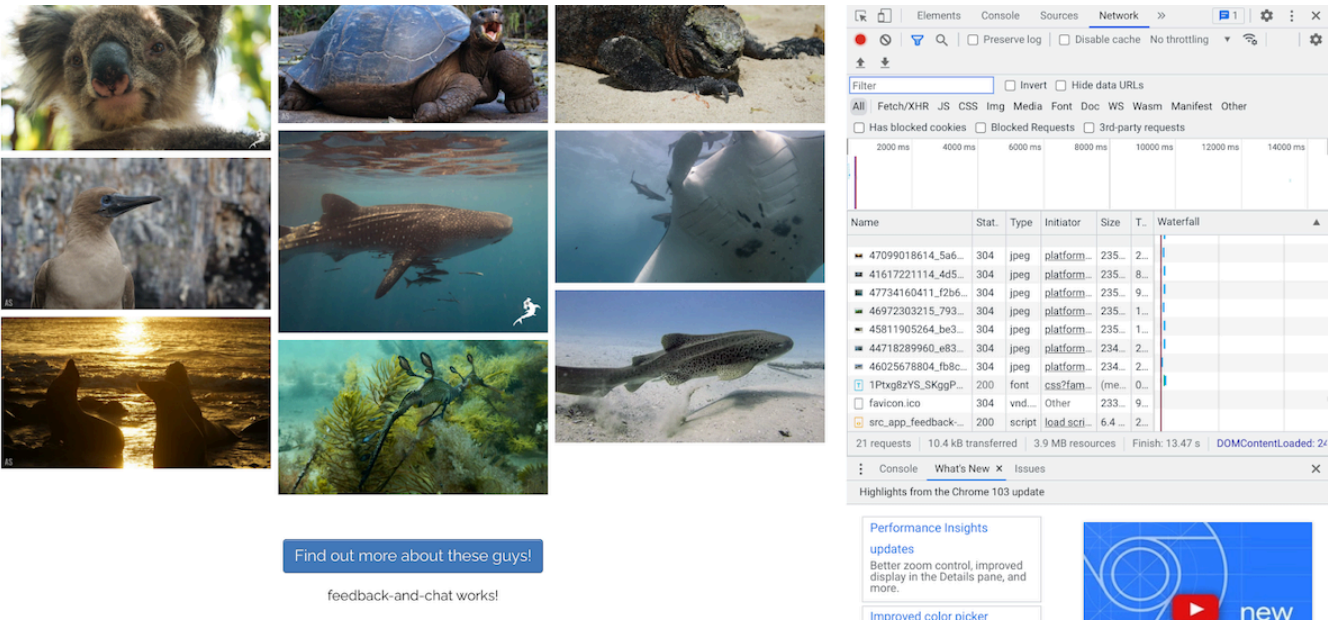
http://localhost:4200

2. 開啟 **Chrome** 開發人員工具，然後選擇「網路」分頁標籤。

3. 重新整理頁面，畫面上會顯示多個檔案，但不會顯示 `feedback-and-chat` 元件。畫面應與下列螢幕截圖相符。



4. 選取「Find out more about these guys!」(進一步瞭解這些人!) 按鈕，即可前往獨立元件。記錄檔應會指出，只有在您點選完整路由時，才會載入元件。捲動至清單結尾，確認元件已載入。畫面應與下列螢幕截圖相符。



步驟 6 · 約 0 分鐘

6. 新增表單的 UI

「傳送意見回饋」表單有三個輸入 UI 欄位，底部有一個按鈕。這三個輸入 UI 欄位分別是全名、寄件者電子郵件地址和留言。

如要支援 UI，請在 `input` 元素中新增 `formControlName` 屬性，並繫結至與三個輸入 UI 欄位相關聯的每個 `contactForm` 表單控制項。

在 HTML 範本中新增表單

在 UI 中新增表單，讓使用者傳送意見回饋。

在程式碼編輯器中完成下列動作。

1. 前往 `feedback-and-chat.component.html` 檔案。
2. 移除現有 HTML。
3. 複製下列程式碼範例中的 HTML。

```

<article>
  <h2>Send me a message or use the chat below to learn some cool facts of the animals
in the photos</h2>
  <section class="container">
    <form ngNativeValidate [formGroup]="contactForm" (ngSubmit)="onSubmit()">
      <div class="row" *ngIf="showMsg">
        <div class="row">
          <p>
            <strong>Thanks for your message!</strong>
          </p>
        </div>
      </div>
      <div class="row">
        <div class="col-25">
          <label for="name">Full Name</label>
        </div>
        <div class="col-75">
          <input type="text" id="fname" placeholder="Your name.."
formControlName="fullname" required>
        </div>
      </div>
      <div class="row">
        <div class="col-25">
          <label for="lemail">Email</label>
        </div>
        <div class="col-75">
          <input type="text" id="lemail" placeholder="Your email.."
formControlName="email" required>
        </div>
      </div>
      <div class="row">
        <div class="col-25">
          <label for="comments">Comments</label>
        </div>
        <div class="col-75">
          <textarea id="subject" formControlName="comments" placeholder="Write
something.." style="height:200px" required></textarea>
        </div>
      </div>
      <div class="row">
        <button type="submit" class="submit">Submit</button>
      </div>
    </form>
  </section>
</article>

```

4. 儲存 `feedback-and-chat.component.html` 檔案。

更新表單的樣式檔案

在程式碼編輯器中完成下列動作。

1. 前往 `feedback-and-chat.component.css` 檔案。
2. 複製下列程式碼範例中的 CSS。

```
/* Style inputs, select elements and textareas */
input[type=text], select, textarea{
  width: 100%;
  padding: 12px;
  border: 1px solid #ccc;
  border-radius: 4px;
  box-sizing: border-box;
  resize: vertical;
}

/* Style the label to display next to the inputs */
label {
  padding: 12px 12px 12px 0;
  display: inline-block;
}

/* Style the submit button */
button {
  background-color: #4479BA;
  color: white;
  padding: 12px 20px;
  border: none;
  border-radius: 4px;
  cursor: pointer;
  float: right;
  font-size: medium;
  font-family: 'Raleway';
}

/* Style the container */
.container {
  border-radius: 5px;
  background-color: #f2f2f2;
  padding: 20px;
  width: 100%;
}

/* Floating column for labels: 25% width */
.col-25 {
  float: left;
  width: 25%;
  margin-top: 6px;
}

/* Floating column for inputs: 75% width */
.col-75 {
  float: left;
  width: 75%;
  margin-top: 6px;
}

/* Clear floats after the columns */
```



```
.row:after {
  content: "";
  display: table;
  clear: both;
}

/* Responsive layout - when the screen is less than 600px wide, make the two columns
stack on top of each other instead of next to each other */
@media screen and (max-width: 600px) {
  .col-25, .col-75, input[type=submit] {
    width: 100%;
    margin-top: 0;
  }
}
```

3. 儲存 `feedback-and-chat.component.css` 檔案。

編譯表單的 UI 更新

在指令列視窗中，完成下列動作。

1. 前往新的 `photo-gallery-app` 專案目錄。
2. 輸入下列 Angular 指令，編譯及執行應用程式，並開啟網路伺服器。

```
ng serve
```

您的應用程式應該無法編譯。別擔心，您需要繫結表單。

3. 目前請先查看下列項目。

- 您可以使用 `formGroup` 屬性繫結，將 `contactForm` 繫結至 `form` 元素和 `ngSubmit` 事件繫結
- `FormGroup` 指令會監聽 `form` 元素發出的提交事件。接著，`FormGroup` 指令會發出 `ngSubmit` 事件，您可以將該事件繫結至 `onSubmit` 回呼函式。
- 在後續步驟中，您將在 `feedback-and-chat.component.ts` 檔案中實作 `onSubmit` 回呼函式

4. 接著，您要繫結表單。

7. 在表單中新增事件處理常式

「傳送意見回饋」表單的 UI 已完成，但缺少互動功能。透過表單處理使用者輸入內容，是許多常見應用程式的基石。

在實際情境中，您需要實作商業邏輯，才能完成下列動作。

1. 剖析與元件相關聯的已算繪 DOM 結構中的使用者輸入內容。
2. 驗證使用者輸入的內容，包括人機驗證或類似機制，避免機器人傳送垃圾內容。
3. 傳送電子郵件至指定電子郵件地址。
4. 向使用者顯示友善訊息。

在本程式碼研究室中，您只會實作下列動作。

1. 剖析與元件相關聯的已算繪 DOM 結構中的使用者輸入內容。
2. 向使用者顯示友善訊息。

請挑戰自己的技能，並實作所有四項動作。

將傳送訊息表單模型新增至元件

在元件類別中建立並新增「提供意見」表單模型。表單模型會決定表單的狀態。FormBuilder 服務提供便利的方法來建立 UI 控制項。

在程式碼編輯器中完成下列動作。

1. 前往 `feedback-and-chat.component.ts` 檔案。
2. 從 `@angular/forms` 套件匯入 `FormBuilder` 服務和 `ReactiveModule` 模組。這項服務提供便利的方法來產生控制項。在下一個步驟中，我們會運用 `inject` 函式，因此也需要從 `@angular/core` 匯入該函式。

```
import { Component, inject, OnInit } from '@angular/core';
import { FormBuilder, ReactiveFormsModule } from '@angular/forms';
```

3. 匯入 `ReactiveFormsModule` 模組。

```
imports: [CommonModule,ReactiveFormsModule],
```

4. 移除下列建構函式。

```
constructor() { }
```

5. 在類別簽章正下方，透過 `inject` 函式插入 `FormBuilder` 服務。

```
private formBuilder = inject(FormBuilder);
```

使用 `FormBuilder` 服務中的 `group` 方法建立表單模型，以便收集使用者的姓名、電子郵件地址和意見。

6. 建立新的 `contactForm` 屬性，並使用 `group` 方法將其設為表單模型。表單模型包含 `name`、`email` 和 `comments` 欄位。

```
contactForm = this.formBuilder.group({
  fullname: '',
  email: '',
  comments: ''
});
```

定義 `onSubmit` 方法來處理表單。在實際情況中，使用者可以透過傳送至指定電子郵件地址的電子郵件訊息，使用 `onSubmit` 方法將意見回饋訊息傳送給攝影師。在本程式碼研究室中，您會顯示使用者輸入內容、使用 `reset` 方法重設表單，並顯示易於理解的成功訊息。

7. 新增 `onSubmit` 方法，並在初始化後將 `showMsg` 變數設為 `true`。

```
showMsg: boolean = false;

onSubmit(): void {
  console.log('Your feedback has been submitted', this.contactForm.value);
  this.showMsg = true;
  this.contactForm.reset();
}
```

`feedback-and-chat.component.ts` 檔案應與下列程式碼範例相符。

```

import { Component, inject, OnInit } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormBuilder, ReactiveFormsModule } from '@angular/forms';

@Component({
  selector: 'app-feedback-and-chat',
  standalone: true,
  imports: [CommonModule, ReactiveFormsModule],
  templateUrl: './feedback-and-chat.component.html',
  styleUrls: ['./feedback-and-chat.component.css']
})
export class FeedbackAndChatComponent {

  showMsg: boolean = false;
  private formBuilder = inject(FormBuilder);

  contactForm = this.formBuilder.group({
    fullname: '',
    email: '',
    comments: ''
  });

  ngOnInit(): void {
  }

  onSubmit(): void {
    console.log('Your feedback has been submitted', this.contactForm.value);
    this.showMsg = true;
    this.contactForm.reset();
  }
}

```

8. 儲存 `feedback-and-chat.component.ts` 檔案。

編譯及顯示「提供意見」表單模型

在指令列視窗中，完成下列動作。

1. 前往新的 `photo-gallery-app` 專案目錄。
2. 輸入下列 Angular 指令，編譯及執行應用程式，並開啟網路伺服器。

```
ng serve
```

3. 開發伺服器應在通訊埠 `4200` 上執行。

在瀏覽器中完成下列動作。

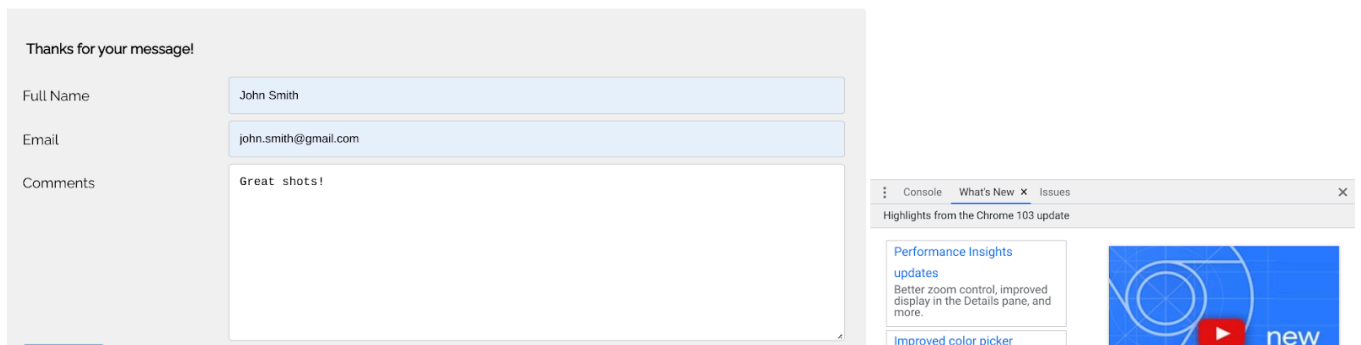
1. 前往下列頁面。

```
http://localhost:4200
```

2. 開啟 **Chrome** 開發人員工具，然後選擇「主控台」分頁。
3. 在「Full Name」(全名)、「Email」(電子郵件) 和「Comments」(留言) 文字方塊中輸入任何值。
4. 選取「提交」按鈕。網頁上應會顯示成功訊息。
5. 確認「控制台」分頁中顯示的值。畫面應與下列螢幕截圖相符。



Send me a message or use the chat below to learn some cool facts of the animals in the photos



您已成功實作程式碼，將「傳送訊息」表單新增至元件 UI，並剖析使用者輸入內容。

6. 接著，您會嵌入即時通訊對話方塊，讓應用程式訪客與 Jimbo 對話。Jimbo 是無尾熊，會告訴你一些有趣的野生動物知識。

8. 新增即時通訊對話

運用對話式介面 (例如 [Dialogflow CX](#) 或類似平台) 的經驗。Dialogflow CX 代理程式是虛擬服務專員，可處理與人類的並行對話。這種自然語言理解模組可以辨識人類語言的細微差異，將對話期間的使用者文字或音訊內容翻譯成應用程式和服務可理解的結構化資料。

系統已為您建立虛擬代理程式範例，在本實驗室中，您需要在獨立元件中新增對話方塊，讓應用程式使用者與機器人互動。為滿足這項需求，您將使用 [Dialogflow Messenger](#)，這項預先建構的整合功能提供可自訂的對話視窗。開啟後，對話方塊會顯示在畫面右下方，並觸發代理程式的預設歡迎意圖。機器人會向使用者問候並發起對話。

以下實作會與使用者分享野生動物的有趣事實。虛擬服務專員的其他實作方式，可能適用於客戶的複雜用途 (例如真人客服中心專員)。許多公司會使用虛擬服務專員做為與公司網站通訊的主要管道。

在元件中新增 Dialogflow Messenger

與表單一樣，只有在載入獨立元件時，才會顯示即時通訊對話方塊。

在程式碼編輯器中完成下列動作。

- 1. 前往 `feedback-and-chat.component.ts` 檔案。
- 2. 複製下列程式碼範例中的 `df-messenger` 元素，並貼到網頁的任何位置。

```
<df-messenger agent-id="762af666-79f7-4527-86c5-9ca06f72c317"
  chat-title="Chat with Jimbo!"
  df-cx="true"
  intent="messenger-welcome-event"
  language-code="en"
  location="us-central1"></df-messenger>
```

選取 Dialogflow Messenger 整合功能時，Dialogflow 會為 `df-messenger` 元素產生屬性。

屬性	詳細資料
agent-id	預先填入。指定 Dialogflow 代理程式的專屬 ID。
chat-title	預先填入。指定要在即時通訊對話方塊頂端顯示的內容。系統會預先填入代理程式名稱，但建議您自訂名稱。
df-cx	表示互動對象為 CX 服務專員。將值設為 <code>true</code> 。
意圖	指定用於在開啟即時通訊對話方塊時觸發第一個意圖的自訂事件。
language-code	指定第一個意圖的預設語言代碼。
位置	指定部署代理程式的區域。

3. 儲存 `feedback-and-chat.component.ts` 檔案。

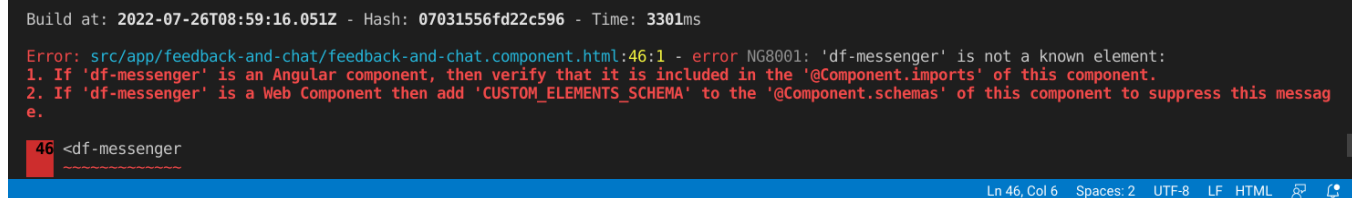
顯示「提供意見」表單模型

在指令列視窗中，完成下列動作。

1. 前往新的 `photo-gallery-app` 專案目錄。
2. 輸入下列 Angular 指令，編譯應用程式。

```
ng serve
```

您的應用程式應該無法編譯。指令列應顯示下列錯誤訊息。



```
Build at: 2022-07-26T08:59:16.051Z - Hash: 07031556fd22c596 - Time: 3301ms
Error: src/app/feedback-and-chat/feedback-and-chat.component.html:46:1 - error NG8001: 'df-messenger' is not a known element:
1. If 'df-messenger' is an Angular component, then verify that it is included in the '@Component.imports' of this component.
2. If 'df-messenger' is a Web Component then add 'CUSTOM_ELEMENTS_SCHEMA' to the '@Component.schemas' of this component to suppress this message.
e.
46 <df-messenger
```

3. 接著新增自訂結構定義。

為獨立元件新增自訂結構定義

`df-messenger` 元素不是已知元素。這是 [自訂網頁元件](#)。Angular 架構的錯誤訊息指出，您必須將 `CUSTOM_ELEMENTS_SCHEMA` 新增至獨立元件，才能隱藏訊息。

在程式碼編輯器中完成下列動作。

1. 前往 `feedback-and-chat.component.ts` 檔案。
2. 匯入 `CUSTOM_ELEMENTS_SCHEMA` 結構定義。

```
import { Component, CUSTOM_ELEMENTS_SCHEMA, inject, OnInit } from '@angular/core';
```

3. 將 `CUSTOM_ELEMENTS_SCHEMA` 新增至結構定義清單。

```
schemas: [CUSTOM_ELEMENTS_SCHEMA]
```

`feedback-and-chat.component.ts` 檔案應與下列程式碼範例相符。

```
import { Component, CUSTOM_ELEMENTS_SCHEMA, inject, OnInit } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormBuilder, ReactiveFormsModule } from '@angular/forms';

@Component({
  selector: 'app-feedback-and-chat',
  standalone: true,
  imports: [CommonModule, ReactiveFormsModule],
  templateUrl: './feedback-and-chat.component.html',
  styleUrls: ['./feedback-and-chat.component.css'],
  schemas: [CUSTOM_ELEMENTS_SCHEMA]
})

...
```

4. df-messenger 網頁元件需要 JavaScript，且只有在載入意見回饋和即時通訊元件時，才應注入 JavaScript。為此，我們將在 `ngOnInit()` 方法中加入相關程式碼，載入啟用 `df-messenger` 元素所需的即時通訊指令碼。

```
ngOnInit() {
  // Load the chat script, which activates the `<df-messenger>` element.
  const script = document.createElement('script');
  script.async = true;
  script.src = 'https://www.gstatic.com/dialogflow-console/fast/messenger-
  cx/bootstrap.js?v=1';
  document.head.appendChild(script);
}
```

5. 儲存 `feedback-and-chat.component.ts` 檔案。

編譯並顯示更新後的即時通訊對話方塊

在指令列視窗中，完成下列動作。

1. 前往新的 `photo-gallery-app` 專案目錄。
2. 輸入下列 Angular 指令，編譯及執行應用程式，並開啟網路伺服器。

```
ng serve
```

開發伺服器應在通訊埠 `4200` 上執行。

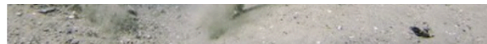
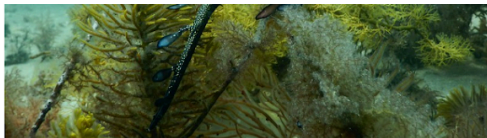
在瀏覽器中完成下列動作。

1. 前往下列頁面。

```
http://localhost:4200
```

頁面底部應該會顯示即時通訊圖示。

2. 選取圖示並與 Jimbo 互動。Jimbo 會提供有趣的動物知識。



Find out more about these guys!

Send me a message or use the chat below to learn some cool facts of the animals

Full Name	Your name..
Email	Your email..
Comments	Write something..
Submit	

Chat with Jimbo!

Hey mate, great to see you. :-) My name is Jimbo and I am the fluffy guy sitting at the top left of this page! The animals you see are endemic to Australia and the Galapagos Islands. Are you keen to learn some fun facts about all of us?

yes

Awesome! Did you know that koalas have fingerprints? We are the only animals other than primates that have them—and just like you, each one of us is an individual! Koalas have to eat A LOT, our main source of food, eucalyptus, is toxic, fibrous, and not very nutritious. I eat about 1/2-1 kilogram of leaves... every night! Think about how you feel after eating a heavy meal, and you will understand why we sleep 18-20 hours a day! How

Ask something...



3. 應用程式可完整運作。

步驟 9 · 約 0 分鐘

9. 將應用程式部署至 Google App Engine

應用程式在本機上執行。在本程式碼研究室的最後一個步驟中，您要將應用程式部署至 Google App Engine。

如要進一步瞭解 Google App Engine，請參閱 [App Engine](#)。

設定 Google App Engine 環境

如果您已符合下列所有條件，請略過後續步驟，直接部署應用程式。

- 建立含有 App Engine 的 Cloud 專案
- 已啟用 Cloud Build API
- 已安裝 Google Cloud CLI

請完成下列動作。

1. 登入 Gmail 或 Google Workspace 帳戶。如果沒有，請[建立 Google 帳戶](#)。
2. 在 Cloud Console 中啟用帳單功能，即可使用 Cloud 資源和 API。詳情請參閱「[建立、修改或關閉自助式 Cloud Billing 帳戶](#)」。
3. 建立 Google Cloud 專案，存放 App Engine 應用程式資源和其他 Google Cloud 資源。詳情請參閱「[建立與管理專案](#)」。
4. 在 Cloud Console 中啟用 Cloud Build API。詳情請參閱「[啟用 API 存取權](#)」。
5. 安裝 Google Cloud CLI 和 gcloud 指令列工具。詳情請參閱「[安裝 gcloud CLI](#)」。
6. 初始化 Google Cloud CLI，並確保 gcloud 已設為使用要部署的 Google Cloud 專案。詳情請參閱「[初始化 gcloud CLI](#)」。

建構獨立應用程式

在指令列視窗中，完成下列動作。

1. 前往新的 `photo-gallery-app` 專案目錄。
2. 輸入下列 Angular 指令，編譯並建立應用程式的正式版。

```
ng build
```

應用程式的正式版會建立在 `photo-gallery-app` 專案目錄的 `dist` 子目錄中。

10. 使用 Express.js 架構部署應用程式

本程式碼研究室的範例程式碼使用 Express.js 架構處理 HTTP 要求。您可以使用偏好的網路架構。

安裝 Express.js 架構

在指令列視窗中，完成下列動作。

1. 輸入下列指令，安裝 Express.js 架構。

```
npm install express --save
```

設定網路伺服器

在程式碼編輯器中完成下列動作。

1. 前往新的 `photo-gallery-app` 專案目錄。
2. 建立新的 `server.js` 檔案。
3. 複製並貼上下列程式碼。

```
const express = require("express");
const path = require("path");

// Running PORT is set automatically by App Engine
const port = process.env.PORT || 3000;
const app = express();

const publicPath = path.join(__dirname, "/dist/photo-gallery-app");

app.use(express.static(publicPath));

app.get("*", (req, res) => {
  res.sendFile(path.join(__dirname + "/dist/photo-gallery-app/index.html"));
});

app.listen(port, () => {
  console.log(`Server is up on ${port}`);
});
```

4. 儲存 `server.js` 檔案。

附加網路伺服器

在程式碼編輯器中完成下列動作。

1. 前往新的 `photo-gallery-app` 專案目錄。
2. 開啟 `package.json` 檔案。
3. 修改啟動指令，在 `server.js` 檔案中執行節點。

```
"name": "photo-gallery-app",
"version": "0.0.0",
"scripts": {
  "ng": "ng",
  "start": "node server.js",
  "build": "ng build",
  "watch": "ng build --watch --configuration development",
  "test": "ng test"
},
```

`start` 屬性是應用程式的進入點。

4. 儲存 `package.json` 檔案。

設定 App Engine

在程式碼編輯器中完成下列動作。

1. 前往新的 `photo-gallery-app` 專案目錄。
2. 建立新的 `app.yaml` 檔案。
3. 複製並貼上下列程式碼。

```
runtime: nodejs16
service: default
```

`app.yaml` 檔案中的資訊會指定 App Engine 的設定。您只需要指定執行階段和服務。

4. 儲存 `app.yaml` 檔案。

更新 gcloud 忽略清單

如要確保系統不會上傳 `node_modules` 目錄，請建立 `.gcloudignore` 檔案。`.gcloudignore` 檔案中列出的檔案不會上傳。

在程式碼編輯器中完成下列動作。

1. 前往新的 `photo-gallery-app` 專案目錄。
2. 建立新的 `.gcloudignore` 檔案。
3. 複製並貼上下列程式碼。

```
# This file specifies files that are *not* uploaded to Google Cloud
# using gcloud. It follows the same syntax as .gitignore, with the addition of
# "#!include" directives (which insert the entries of the given .gitignore-style
# file at that point).
#
# For more information, run:
#   $ gcloud topic gcloudignore
#
.gcloudignore
# If you would like to upload your .git directory, .gitignore file or files
# from your .gitignore file, remove the corresponding line
# below:
.git
.gitignore

# Node.js dependencies:
node_modules/
```

4. 儲存 `.gcloudignore` 檔案。

初始化應用程式

部署應用程式前，請先使用專案初始化應用程式，然後選擇相關聯的區域。

在指令列視窗中，完成下列動作。

1. 前往新的 `photo-gallery-app` 專案目錄。
2. 輸入下列指令來初始化應用程式。

```
gcloud app create --project=[YOUR_PROJECT_ID]
```

3. 按照提示選擇要放置 App Engine 應用程式的地區。

部署應用程式

在指令列視窗中，完成下列動作。

1. 前往新的 `photo-gallery-app` 專案目錄。
2. 輸入下列指令來部署應用程式。

```
gcloud app deploy --project=[YOUR_PROJECT_ID]
```

3. 在提示中確認動作。gcloud 部署指令成功完成後，畫面會顯示存取應用程式的網址。
4. 輸入下列指令，在瀏覽器中開啟新分頁。

```
gcloud app browse
```

```
alessiasacchi@penguin:~/angular/photo-gallery-app$ gcloud app deploy --project=diaologflow-cx-playground
Services to deploy:

descriptor:          [/home/alessiasacchi/angular/photo-gallery-app/app.yaml]
source:              [/home/alessiasacchi/angular/photo-gallery-app]
target project:      [diaologflow-cx-playground]
target service:      [default]
target version:      [20220726t114400]
target url:          [https://diaologflow-cx-playground.uc.r.appspot.com]
target service account: [App Engine default service account]

Do you want to continue (Y/n)? Y

Beginning deployment of service [default]...

[= Uploading 1 file to Google Cloud Storage =]

File upload done.
Updating service [default]...done.
Setting traffic split for service [default]...done.
Deployed service [default] to [https://diaologflow-cx-playground.uc.r.appspot.com]

You can stream logs from the command line by running:
$ gcloud app logs tail -s default

To view your application in the web browser run:
$ gcloud app browse
```

11. 恭喜

恭喜！您已使用 Angular v14 建立範例相片庫，並成功部署至 App Engine。

您試用了獨立元件功能和延遲載入功能。您已建構以表單為基礎的傳送訊息功能，可提供意見和留言。您也已使用 Dialogflow Messenger，成功新增與 Dialogflow CX 虛擬服務專員的即時通訊對話。做得好！

後續步驟

您已完成基本應用程式，現在請運用下列想法進行改良。

- 表單實際上不會傳送意見回饋，請重構表單，加入傳送電子郵件的業務邏輯。
- 在實際情況中，您應驗證使用者輸入內容，並加入人機驗證或類似機制，避免機器人濫發垃圾內容
- 建立新虛擬服務專員，並瞭解如何在 Dialogflow CX 中設計對話流程

繼續試用 Angular 架構，盡情享受開發樂趣。

清除及刪除 Cloud 專案

您可以保留 Cloud 專案，也可以刪除專案，這樣系統就會停止對專案使用的所有資源收取費用。

在瀏覽器中完成下列動作。

1. 登入 Gmail 或 Google Workspace 帳戶。
 2. 在 **Google Cloud** 控制台中，選取「IAM & Admin Settings」(IAM 與管理設定) 按鈕。
 3. 在「IAM 與管理」頁面上，選取「管理資源」分頁標籤。
 4. 在「管理資源」頁面中，前往並選擇要刪除的專案。選取「刪除」按鈕，開啟對話方塊。
 5. 在對話方塊視窗中輸入專案 ID，然後選取「Shut down」(關閉) 按鈕刪除專案。
-